Digital VLSI Assignment: 4-bit Array Multiplier

RTL to GDSII with Open-Source Tools

Kavya Veer (2024112002) & Dhanika Kothari (2024102011)

Repository: `https://github.com/heyitscass/mult4_2024102011_2024112002`

---

1 Introduction

This report covers Parts A-D of the 4-bit array multiplier assignment: RTL design and verification, synthesis, static timing analysis, and physical design, all targeting the SkyWater SKY130 process (sky130_fd_sc_hd standard cells). The multiplier takes two 4-bit inputs A and B and produces a registered 8-bit output, with 2 clock cycles of latency. After synthesis the design came to 76 cells, and Part C's timing analysis said it could run at 384.62 MHz. Once we actually placed and routed it (Part D), the real achievable clock turned out to be much slower, a final period of 8.0 ns was needed. Section 5 explains why.

2 Design and verification (Part A)

2.1 Circuit structure

Multiplying two 4-bit numbers is shift-and-add. For every bit of B, we AND it with all 4 bits of A to get a partial product row, 16 AND gates total for 4 rows. Row 0 is just those AND gates, no adder needed, and it directly gives the lowest output bit p[0]. Rows 1-3 are each a 4-bit ripple-carry adder (4 full adders per row, 12 total) that adds in the next partial product row plus whatever the row above passed down, and gives out the next finished output bit each time. The last row also gives the top 4 output bits directly.

Inputs get registered on one clock edge, output on the next edge after that, so 16 flip-flops total (4+4+8) and 2 cycles of latency end to end.

2.2 Files

- **rtl/full_adder.v**, a single 1-bit full adder, two assign statements (sum = A $\oplus$ B $\oplus$ Cin, Cout = majority(A, B, Cin)).
- **rtl/mult_array.v**, the full multiplier: input registers, the 16-gate AND array, three rows of full adders, output register. Instantiates full_adder 12 times. No `*` operator anywhere.
- **tb/tb_mult.v**, exhaustive, self-checking testbench, all 256 input combinations, checked against a reference computed with `*` (the only place that operator is used anywhere in the project).

2.3 Verification results

The testbench runs a 10 ns clock, holds reset low for the first 2 cycles, then steps through all 256 combinations, waiting the full 2-cycle latency before checking the output.

```
PASS: 256 vectors checked, 0 errors
```

Final summary line from the exhaustive testbench run.

To make sure the testbench actually catches bugs and isn't just always printing PASS, we broke full_adder.v on purpose (dropped the carry-in term from the sum equation) and reran it:

```
FAIL: 256 vectors checked, 95 errors
```

Testbench correctly detects the injected fault. full_adder.v was fixed again right after this check.

2.4 A timing bug we found

The first version of the testbench changed inputs at the exact same instant as a clock edge. That happened to work fine in RTL simulation (we got lucky with how the simulator orders events at that instant), but when we simulated the actual synthesized gate-level netlist (Section 3), it failed, every result came out exactly one vector behind.

The fix: change inputs right after a falling clock edge instead of exactly on a rising edge, and add a 1ns delay before reading the output so the register's update has definitely settled. After that fix, RTL and gate-level simulation agree exactly, no luck involved either way.

2.5 Hand-worked examples

Each of us hand-computed A × B using our own roll number (A = last digit of roll number + 5, B = 7, or 9 if A would equal 7), in the same partial-product layout as the assignment brief, and found the same multiplication in the simulated waveform.

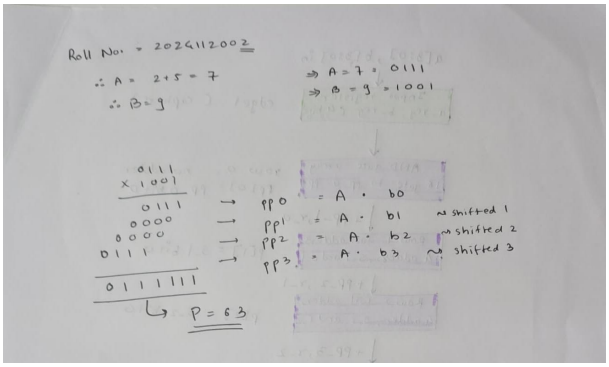


Figure 1: Kavya's hand-worked example

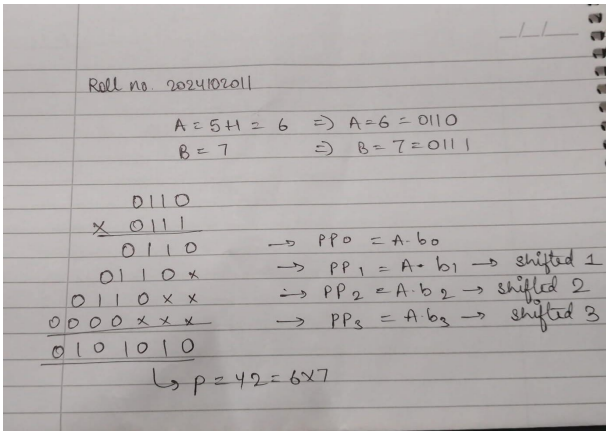


Figure 2: Dhanika's hand-worked example

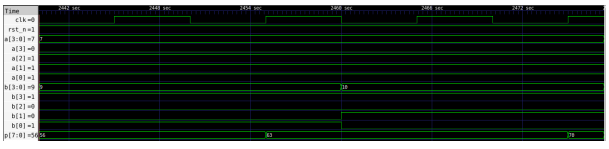


Figure 3: Kavya's GTKWave: p holds 56 (previous vector, 7×8) for one clock period after b settles to 9, then updates to 63 (7×9) and holds until b moves on to 10, showing the 2-cycle latency

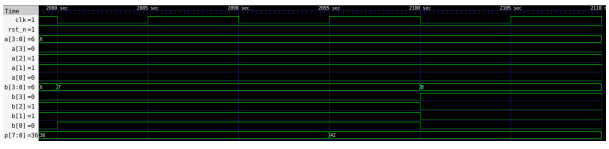


Figure 4: Dhanika's GTKWave: p holds 36 (previous vector, 6×6) for one clock period after b settles to 7, then updates to 42 (6×7) and holds until b moves on to 8, showing the 2-cycle latency

2.6 Block diagram

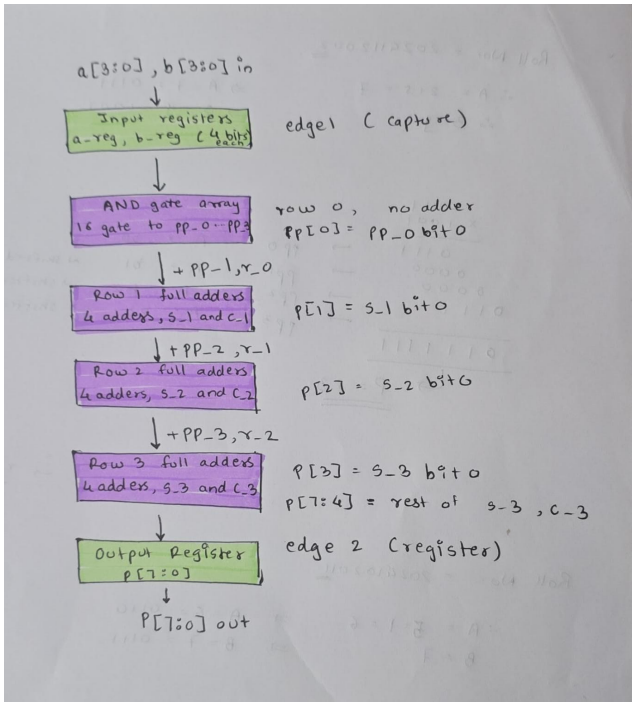


Figure 5: Block Diagram

3 Logic synthesis (Part B)

Synthesized to SKY130 standard cells with Yosys, using synth/synth.tcl:

```
read_verilog rtl/full_adder.v rtl/mult_array.v
synth -top mult_array -flatten
dfflibmap -liberty $lib
abc -liberty $lib
opt_clean -purge
tee -o synth/mult_array_stat.rpt stat -liberty $lib
write_verilog -noattr synth/mult_array_netlist.v
```

Synthesis script, run with: yosys -c synth/synth.tcl

3.1 Results

Metric	Value
Total number of cells	76
Number of flip-flops	16
Total cell area	814.531 μm^2
Flip-flop area	400.384 μm^2
Flip-flop area (% of total)	49.16%
3 most-used cell types	dfrtp_1 (16), nand2_1 (11), xnor2_1 (5)
Netlist simulation result	PASS: 256 vectors checked, 0 errors

Table 1: Synthesis results for mult_array, from synth/mult_array_stat.rpt.

3.2 Netlist verification

Re-ran the same testbench against the synthesized netlist, linked against the real SKY130 cell models, to make sure synthesis didn't change behaviour:

```
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls tb/tb_mult.v \
  $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/mult_array_netlist.v
vvp gls
```

```
PASS: 256 vectors checked, 0 errors
```

Gate-level netlist simulation result.

4 Static timing analysis (Part C)

Checked timing with OpenSTA against the typical process corner only (tt_025C_1v80), using sta/sta.tcl:

```
read_liberty $lib
read_verilog synth/mult_array_netlist.v
link_design mult_array
create_clock -name clk -period <T> [get_ports clk]
set_input_delay  1.0 -clock clk [all_inputs]
set_output_delay 1.0 -clock clk [all_outputs]
report_worst_slack -max
report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt
```

Timing analysis script, run with: sta sta.tcl

4.1 Finding T_{min} and F_{max}

First ran at T = 10 ns to get the worst slack (WNS). From there, $T_{min} = T - WNS$ and $F_{max} = 1000/T_{min}$.

Metric	Value
Worst slack at T = 10 ns	7.40 ns
T_{min} = T - WNS = 10 - 7.40	2.60 ns
F_{max} = 1000 / T_{min}	384.62 MHz
Slack at T = 2.61 ns (rerun)	0.007 ns (MET)
Critical path start point	flip-flop at the end of the row-1 adder chain (netlist cell _124_)
Critical path end point	flip-flop in the row-3 adder chain (netlist cell _118_)
Cells on the critical path	10 (8 combinational cells + 2 flip-flops at either end)

Table 2: Timing results for mult_array, from sta/mult_array_timing.rpt. T_{min} rounds to 2.60 ns, but that alone leaves a 3ps violation, so the confirming rerun used 2.61 ns.

4.2 Critical path

These cell names (and2_0, o21ai_0, etc.) are generic SKY130 gates that ABC picked during synthesis, they don't map one-to-one back to our 12 full_adder instances, since synthesis flattens the whole design and re-optimizes it against the cell library. Still, 8 logic stages between the two flip-flops is close to the assignment's own estimate of $2N = 8$ full-adder delays for N = 4. Makes sense, the critical path basically runs straight through the ripple-carry chain.

5 Physical design (Part D)

Used LibreLane (the OpenLane 2 successor) with pnr/config.json, fixed 150 μm × 150 μm die:

```
{
  "DESIGN_NAME": "mult_array",
  "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/mult_array.v
     "],
  "CLOCK_PORT": "clk",
  "CLOCK_PERIOD": 8.0,
  "FP_SIZING": "absolute",
  "DIE_AREA": [0, 0, 150, 150]
}
```

Final pnr/config.json, run with: librelane config.json

5.1 Why 4 attempts were needed

We started with CLOCK_PERIOD = 3.5 ns (1.2 × T_{min} from Part C, rounded up). That assumed the typical corner was good enough, it isn't, because LibreLane's signoff timing checks several process corners, including a slow/hot/low-voltage one (ss_100C_1v60) where every cell is just slower. At 3.5 ns that corner failed setup badly (slack -2.154 ns). We tried 6.0 ns and 6.5 ns next and the violation barely moved (it was actually slightly worse at 6.5 ns than 6.0 ns), so instead of inching up further we jumped to 8.0 ns, which passed cleanly everywhere.

Attempt	**CLOCK_PERIOD**	**Worst setup slack**	**DRC / LVS**
1	3.5 ns	-2.154 ns (slow corner)	0 / 0
2	6.0 ns	-0.107 ns (slow corner)	0 / 0
3	6.5 ns	-0.136 ns (slow corner)	0 / 0
4	8.0 ns	+1.364 ns	0 / 0

Table 3: LibreLane attempts. DRC and LVS were clean from the first attempt, only setup timing at the slow corner needed fixing.

5.2 Final results

Metric	**Value**
CLOCK_PERIOD used	8.0 ns
Number of runs needed	4
Core area	17759.5 μm^2
Cells after synthesis (Part B)	76
Cells after place and route	1895
Kinds of cells added	clock buffers, timing-repair buffers, fill cells, tap cells, decap cells
Wirelength	2362 μm
Setup slack	+1.364 ns
Post-route F_{max} = 1000/(8.0 - 1.364)	150.70 MHz
F_{max} from Part C	384.62 MHz
DRC errors	0
LVS errors	0

Table 4: Final Part D results, from pnr/metrics.json.

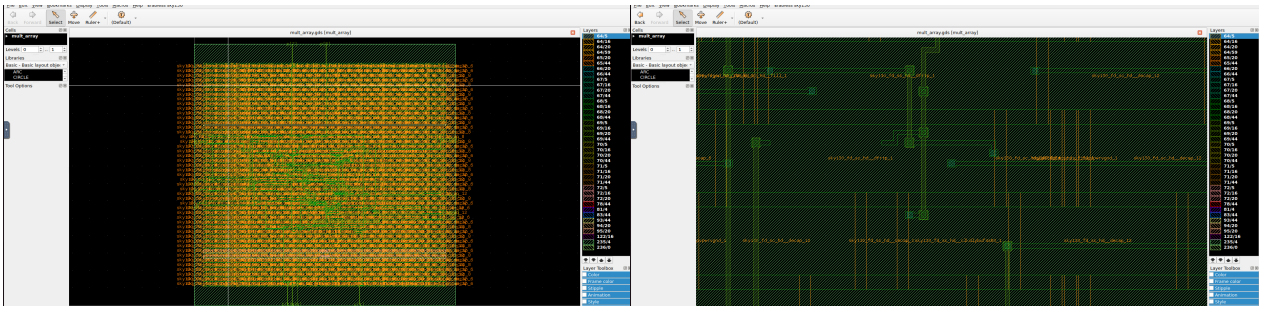


Figure 6: Whole die

Figure 7: Zoomed view

Figure 8: KLayout screenshots

6 Answers to questions

Q1. How many cells did synthesis produce for the logic, and why isn't it simply 16 + 12?

76 cells, not 28 (16+12). `abc` doesn't keep our AND gates and full adders as separate blocks, it flattens the whole design and re-maps the logic onto whatever SKY130 cells fit best and use the least area. A full adder can get absorbed into one bigger optimized gate instead of staying as its own block. The final cell count is what the optimizer picked, not a gate-for-gate copy of our RTL.

Q2. What is the critical path, and how many cells are on it?

10 cells: 8 combinational gates (and2_0, o21ai_0, a32oi_1, o311ai_0, a211oi_1, o21a_1, o31ai_1, xnor3_1) between the start and end flip-flops. That lines up with the $2N = 8$ estimate for N = 4 from the assignment. The gate names don't match our original full_adder instances, since synthesis re-optimizes everything, but the path still clearly runs through the ripple-carry chain.

Q3. What percentage of the total cell area is flip-flops, and why is this large for such a small design?

49.16% of the area is flip-flops (400.384 out of 814.531 μm^2). We have 60 combinational cells doing simple AND/XOR/adder logic, versus 16 flip-flops, and each flip-flop is a bigger, more complex structure than a basic gate, it needs two internal latches plus clock and reset wiring, so it ends up several times the area of something like a NAND gate. In a design this small, that fixed register cost is almost half the chip.

Q4. Why is the post-route F_max lower than the F_max from Part C? Which kinds of cells did the flow add, and why is each needed?

150.70 MHz post-route vs 384.62 MHz from Part C. Most of the gap is because Part C only checked the typical corner, and the real Part D signoff also checks the slow corner (ss_100C_1v60), where cells are slower, we had to relax the clock period all the way to 8.0 ns just to pass there. Real routing parasitics that Part C's idealized analysis doesn't model add a bit more delay on top of that.

The flow also adds cells that don't exist right after synthesis. Clock buffers get added so the clock reaches all 16 flip-flops with decent skew. The resizer inserts extra buffers to fix slew or setup problems on specific nets. Tap cells go in for DRC and to prevent latch-up. Decap cells stabilize the power supply. Fill cells do nothing electrically but are needed to meet the foundry's metal density rules. None of this shows up in a synthesis-only netlist, since it's all about actual physical placement and routing, which doesn't exist at that stage yet.

Q5. What does a DRC check verify, and what does an LVS check verify? Why must both pass before a chip can be manufactured?

DRC checks that the actual shapes on each mask layer follow the foundry's manufacturing rules, minimum width, minimum spacing, and so on. Break these and the chip might just fail to print correctly: shorts, opens, bad transistors.

LVS checks that the layout, when you extract the devices and connections back out of it, actually matches the netlist we meant to build, same transistors, same connections, nothing missing or extra.

Both need to pass because they check different things. DRC alone doesn't tell you the circuit is wired correctly, you could have a clean, manufacturable layout that's still the wrong circuit. LVS alone doesn't tell you it can be built, you could have exactly the right circuit laid out in a way that breaks spacing rules. You need both before you can trust the chip will actually work once it's fabricated.

Q6. Using the formulas in Section 2, estimate how the number of full adders and the critical path delay would change for an 8-bit multiplier.

An N-bit multiplier needs $N(N-1)$ full adders, with a critical path around $2N$ full-adder delays. For us, N = 4: $4 \times 3 = 12$ adders, $2 \times 4 = 8$ delay stages, matches what we actually got.

For N = 8: $8 \times 7 = 56$ adders, 4.7× more than our 12. But only $2 \times 8 = 16$ delay stages, just 2× more than our 8. The adder count grows roughly with N^2 (N rows, each needing N adders), while the critical path only grows linearly with N, since it's just the length of one carry chain. So doubling the bit width costs a lot more in area than it costs in speed, part of why real multipliers move away from plain ripple-carry arrays once N gets large.

7 Conclusion

All four parts are done: RTL designed and exhaustively tested (256/256 vectors, plus the fault-injection check), synthesized to 76 cells with the same testbench re-verified at gate level, timed at 384.62 MHz at the typical corner, and placed and routed to a clean DRC/LVS layout. The real-world clock period ended up at 8.0 ns, not the roughly 3.5 ns Part C's number suggested, because of the slow-corner timing LibreLane checks and Part C doesn't. Flip-flops taking up about half the area even in a design this small, and the gap between typical-corner and worst-corner timing, were the two biggest things we didn't expect going in.

8 Individual Contribution

Task	**Kavya Veer (2024112002)**	**Dhanika Kothari (2024102011)**
Part A - RTL	Wrote `full_adder.v` and `mult_array.v`	Wrote `tb_mult.v`, ran simulation
Part B - Synthesis	Wrote and ran `synth.tcl`, ran GLS	Reviewed results
Part C - Timing	Reviewed results	Wrote and ran `sta.tcl`, computed T_{min}, F_{max}
Part D - Physical design	Wrote `config.json`, ran LibreLane (4 attempts)	Analysed metrics, took KLayout screenshots
Report	Sections 3, 5 (Part B, D)	Sections 4, 6 (Part C, questions)

Table 5: Individual contributions